SEMESTER S1

ALGORITHMIC THINKING WITH PYTHON

(Common to All Branches)

Course Code	**UCEST105**	**CIE Marks**	40
Teaching Hours/Week(L: T:P: R)	3:0:2:0	**ESE Marks**	60
Credits	4	**Exam Hours**	2 Hrs. 30 Min
Prerequisites (if any)	None	**Course Type**	Theory

Course Objectives:

1. To provide students with a thorough understanding of algorithmic thinking and its practical applications in solving real-world problems.
2. To explore various algorithmic paradigms, including brute force, divide-and-conquer, dynamic programming, and heuristics, in addressing and solving complex problems.

SYLLABUS

Module No.	Syllabus Description	Contact Hours
1	PROBLEM-SOLVING STRATEGIES:- Problem-solving strategies defined, Importance of understanding multiple problem-solving strategies, Trial and Error, Heuristics, Means-Ends Analysis, and Backtracking (Working backward). THE PROBLEM-SOLVING PROCESS:- Computer as a model of computation, Understanding the problem, Formulating a model, Developing an algorithm, Writing the program, Testing the program, and Evaluating the solution. ESSENTIALS OF PYTHON PROGRAMMING:- Creating and using variables in Python, Numeric and String data types in Python, Using the math module, Using the Python Standard Library for handling basic I/O - print, input, Python operators and their precedence.	**7**

2	ALGORITHM AND PSEUDOCODE REPRESENTATION:- Meaning and Definition of Pseudocode, Reasons for using pseudocode, The main constructs of pseudocode - Sequencing, selection (if-else structure, case structure) and repetition(for, while, repeat-until loops), Sample problems * FLOWCHARTS** :- Symbols used in creating a Flowchart - start and end, arithmetic calculations, input/output operation, decision (selection), module name (call), for loop (Hexagon), flow-lines, on-page connector, off-page connector. ** - Evaluate an expression, **d=a+b*c**, find simple interest, determine the larger of two numbers, determine the smallest of three numbers, determine the grade earned by a student based on KTU grade scale (using if-else and case structures), print the numbers from 1 to 50 in descending order, find the sum of n numbers input by the user (using all the three loop variants), factorial of a number, largest of n numbers (**Not to be limited to these exercises. More can be worked out if time permits**).* *** **Only for visualizing the control flow of Algorithms. The use of tools like RAPTOR (https://raptor.martincarlisle.com/) is suggested. Flowcharts for the sample problems listed earlier may be discussed***	**9**
3	SELECTION AND ITERATION USING PYTHON:- if-else, elif, for loop, range, while loop. Sequence data types in Python - list, tuple, set, strings, dictionary, Creating and using Arrays in Python (using ***Numpy*** library). DECOMPOSITION AND MODULARIZATION* :- Problem decomposition as a strategy for solving complex problems, Modularization, Motivation for modularization, Defining and using functions in Python, Functions with multiple return values	**10**

	RECURSION:- Recursion Defined, Reasons for using Recursion, The Call Stack, Recursion and the Stack, Avoiding Circularity in Recursion, *Sample problems - Finding the **n**th Fibonacci number, greatest common divisor of two positive integers, the factorial of a positive integer, adding two positive integers, the sum ofdigits of a positive number* **. * *The idea should be introduced and demonstrated using Merge sort, the problem of returning the top three integers from a list of **n>=3** integers as examples.* (***Not tobe limited to these two exercises. More can be worked out if time permits***). ** ***Not to be limited to these exercises. More can be worked out if time permits***.	
4	COMPUTATIONAL APPROACHES TO PROBLEM-SOLVING (*Introductory diagrammatic/algorithmic explanations only. Analysis not required*) :- Brute-force Approach - - *Example: Padlock, Password guessing* Divide-and-conquer Approach - - *Example: The Merge Sort Algorithm* - Advantages of Divide and Conquer Approach - Disadvantages of Divide and Conquer Approach Dynamic Programming Approach - *Example: Fibonacci series* - Recursion vs Dynamic Programming Greedy Algorithm Approach - *Example: Given an array of positive integers each indicating the completion time for a task, find the maximum number of tasks that can becompleted in the limited amount of time that you have.* - Motivations for the Greedy Approach - Characteristics of the Greedy Algorithm - Greedy Algorithms vs Dynamic Programming Randomized Approach - *Example 1: A company selling jeans gives a coupon for each pair of jeans. There are n different coupons. Collecting **n** different coupons would give you free jeans. How many jeans do you expect to buy before getting a free one?*	**10**

	*- Example 2: **n** people go to a party and drop off their hats to a hat-check person. When the party is over, a different hat-check person is on duty andreturns the **n** hats randomly back to each person. What is the expected number of people who get back their hats?* - Motivations for the Randomized Approach	

Course Assessment Method
(CIE: 40 marks, ESE: 60 marks)

Continuous Internal Evaluation Marks (CIE):

Attendance	Continuous Assessment (Accurate Execution of Programming Tasks)	Internal Examination-1 (Written Examination)	Internal Examination-2 (Written Examination)	Internal Examination- 3 (Lab Examination)	Total
5	5	10	10	10	40

End Semester Examination Marks (ESE)

In Part A, all questions need to be answered and in Part B, each student can choose any one fullquestion out of two questions

Part A	Part B	Total
• 2 Questions from each module. • Total of 8 Questions, each carrying 3 marks **(8x3 =24marks)**	• Each question carries 9 marks. • Two questions will be given from each module, out of which 1 question should be answered. • Each question can have a maximum of 3 sub divisions. **(4x9 = 36 marks)**	**60**

Course Outcomes (COs)

At the end of the course students should be able to:

Course Outcome		Bloom's Knowledge Level (KL)
CO1	Utilize computing as a model for solving real-world problems.	**K2**
CO2	Articulate a problem before attempting to solve it and prepare a clear and accurate model to represent the problem.	**K3**
CO3	Utilize effective algorithms to solve the formulated models and translate algorithms into executable programs.	**K3**
CO4	Interpret the problem-solving strategies, a systematic approach to solving computational problems, and essential Python programming skills	**K3**

Note*: K1- Remember, K2- Understand, K3- Apply, K4- Analyse, K5- Evaluate, K6- Create*

CO-PO Mapping Table:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	3									3
CO2	3	3	3									3
CO3	3	3	3									3
CO4	3	3	3									3

Reference Books				
Sl. No	**Title of the Book**	**Name of the Author/s**	**Name of the Publisher**	**Edition and Year**
1	Problem solving & programming concepts	Maureen Sprankle, Jim Hubbard	Pearson	9/e, 2011
2	How to Solve It: A New Aspect of Mathematical Method	George Pólya	Princeton University Press	2/e, 2015
3	Creative Problem Solving: An Introduction	Donald Treffinger., Scott Isaksen, Brian Stead-Doval	Prufrock Press	4/e,2005
4	Psychology (Sec. Problem Solving.)	Spielman, R. M., Dumper, K., Jenkins, W., Lacombe, A., Lovett, M., & Perlmutter, M	H5P Edition	1/e, 2021
5	Computational Thinking: A Primer for Programmers and Data Scientists	G Venkatesh Madhavan Mukund	Mylspot Education Services Pvt Ltd	1/e, 2020
6	Computer Arithmetic Algorithms	Koren, Israel	AK Peters/CRC Press	2/e, 2001
7	Python for Everyone	Cay S. Horstmann, Rance D. Necaise	Wiley	3/e, 2024
8	Introduction to Computation and Programming using Python	Guttag John V	PHI	2/e., 2016

Video Links (NPTEL, SWAYAM…)	
Module No.	**Link ID**
1	https://opentextbc.ca/h5ppsychology/chapter/problem-solving/
2	https://onlinecourses.nptel.ac.in/noc21_cs32/preview

1. Continuous Assessment (5 Marks)

Accurate Execution of Programming Tasks

- Correctness and completeness of the program
- Efficient use of programming constructs
- Handling of errors
- Proper testing and debugging

2. Evaluation Pattern for Lab Examination (10 Marks)

1. **Algorithm (2 Marks)**

   Algorithm Development: Correctness and efficiency of the algorithm related to the question.

2. **Programming (3 Marks)**

   Execution: Accurate execution of the programming task.

3. **Result (3 Marks)**

   Accuracy of Results: Precision and correctness of the obtained results.

4. **Viva Voce (2 Marks)**

Proficiency in answering questions related to theoretical and practical aspects of the subject.

Sample Classroom Exercises:

1. Identify three ill-defined problems and well-defined problems
2. Identify five use cases for Trial and error, Heuristics, backtracking, and Means-ends analysis.
3. Use a diagram to solve the Tower of Hanoi for three pegs with the minimum number of moves.
4. Evaluate different algorithms discussed earlier based on their efficiency by counting the number of steps.
5. A recursive function that takes a number and returns the sum of all the numbers from zero to that number.
6. A recursive function that takes a number as an input and returns the factorial of that number.
7. A recursive function that takes a number '**n**' and returns the nth Fibonacci number.
8. A recursive function that takes an array of numbers as input and returns the product of all the numbers in the array.
9. A program to reverse the contents of an **1D** array without using a second array.
10. To register for the end-semester examination, you need to log into the University portal with your credentials. Write a program to validate the credentials. Assume that the usernames are stored in an array of strings called **USERNAME** and the corresponding passwords are stored in another array of strings called **PASSWORD** such that **password[i]** is the password for the user **username[i]**.
11. You are given a list and your task is to divide it to make two smaller lists. The sublists should be made from alternate elements in the original list. So if the original list is **{5,1,4,12,6}**, then one sublist should be **{5,4,6}** and the other should be **{1,12}**.
12. A program that takes three points in a 2D plane and determines whether they are collinear. Two pairs of points are collinear if they have the same slope.

LAB Experiments:

1. Simple desktop calculator using Python. *Only the five basic arithmetic operators*.
2. Create, concatenate, and print a string and access a sub-string from a given string.
3. Familiarize time and date in various formats (Eg. "Thu Jul 11 10:26:23 IST 2024").
4. Write a program to create, append, and remove lists in Python using NumPy.
5. Program to find the largest of three numbers.
6. Convert temperature values back and forth between Celsius (c), and Fahrenheit (f). [Formula: c/5 = f-32/9]
7. Program to construct patterns of stars (*), using a nested for loop.
8. A program that prints prime numbers less than ***N***.